



CineWatch – LLD: Member 4 (ML Engineer, Content-Based)



Role: Member 4 — ML Engineer (Content-Based)

Responsibilities: Mood Selector Engine · Exploratory Data Analysis ·
`content_engine.py`

Tech Stack: Python · TF-IDF · Cosine Similarity · Pandas · Scikit-learn

1. Module Overview

This module powers the **content-based recommendation engine** for CineWatch. Given a user's mood or a selected movie, it returns a ranked list of similar movies using textual feature similarity. It operates fully offline (no collaborative data needed) and is consumed by the backend API.

2. File & Folder Structure

```
CineWatch/  
├─ ml/  
│   ├── content_engine.py      # Core recommendation engine  
│   ├── mood_selector.py      # Mood → genre/tag mapping  
│   └─ eda.ipynb              # Exploratory Data Analysis  
notebook  
│   └─ preprocess.py          # Data cleaning & feature en  
gineering  
│   └─ models/  
│       ├── tfidf_matrix.pkl   # Serialized TF-IDF matrix  
│       └─ cosine_sim.pkl     # Precomputed cosine similar
```

```

ity matrix
|   └─ data/
|       └─ movies_raw.csv      # Raw dataset (e.g., TMDb /
IMDb)
|       └─ movies_clean.csv   # Cleaned & feature-engineer
ed dataset

```

3. Data Flow

```

Raw Dataset (CSV)
  |
  ▼
preprocess.py
- Drop nulls
- Combine features (genre + keywords + cast + director + ov
erview)
- Lowercase + strip
  |
  ▼
content_engine.py
- TF-IDF Vectorizer → sparse matrix
- Cosine Similarity matrix (n×n)
- Serialize → .pkl files
  |
  ▼
Backend API call
- Input: movie_title OR mood tag
- Output: Top-N similar movies (ranked list)

```

4. Low-Level Design: `content_engine.py`

4.1 Class / Function Breakdown

```

build_feature_soup(df: pd.DataFrame) → pd.Series

```

- Combines multiple metadata columns into a single text blob per movie
- Columns combined: `genres`, `keywords`, `cast`, `director`, `overview`
- Applies lowercasing, whitespace normalization
- Returns a Series of combined strings

```
def build_feature_soup(df):
    df['soup'] = (
        df['genres'].fillna('') + ' ' +
        df['keywords'].fillna('') + ' ' +
        df['cast'].fillna('') + ' ' +
        df['director'].fillna('') + ' ' +
        df['overview'].fillna('')
    )
    return df['soup'].str.lower().str.strip()
```

`fit_tfidf(soup: pd.Series) → (TfidfVectorizer, sparse_matrix)`

- Initializes `TfidfVectorizer(stop_words='english')`
- Fits and transforms the soup corpus
- Returns vectorizer + resulting TF-IDF matrix

```
def fit_tfidf(soup):
    tfidf = TfidfVectorizer(stop_words='english')
    tfidf_matrix = tfidf.fit_transform(soup)
    return tfidf, tfidf_matrix
```

`compute_cosine_similarity(tfidf_matrix) → np.ndarray`

- Computes pairwise cosine similarity across all movies
- Uses `sklearn.metrics.pairwise.linear_kernel` (faster than `cosine_similarity` for sparse matrices)
- Output shape: `(n_movies, n_movies)`

```
from sklearn.metrics.pairwise import linear_kernel

def compute_cosine_similarity(tfidf_matrix):
    return linear_kernel(tfidf_matrix, tfidf_matrix)
```

```
get_recommendations(title: str, df: pd.DataFrame, cosine_sim: np.ndarray, top_n=10) → list[dict]
```

- Looks up the row index of the queried movie
- Retrieves its similarity scores from the cosine matrix
- Sorts scores descending, excludes self, returns top-N titles with scores

```
def get_recommendations(title, df, cosine_sim, top_n=10):
    idx_map = pd.Series(df.index, index=df['title'].str.lower())
    idx = idx_map[title.lower()]
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:top_n+1] # exclude self
    movie_indices = [i[0] for i in sim_scores]
    results = df.iloc[movie_indices][['title', 'genres', 'overview']].copy()
    results['score'] = [round(s[1], 4) for s in sim_scores]
    return results.to_dict(orient='records')
```

```
save_artifacts(tfidf_matrix, cosine_sim, path='ml/models/')
```

- Serializes both the TF-IDF matrix and cosine similarity matrix using `pickle`
- Called once after training; artifacts loaded at API startup

```
load_artifacts(path='ml/models/') → (tfidf_matrix, cosine_sim)
```

- Loads pre-serialized matrices at server startup for zero-latency inference

5. Low-Level Design: `mood_selector.py`

5.1 Mood → Feature Mapping Table

Mood	Mapped Genres / Keywords
Happy	comedy, feel-good, animated, family
Sad	drama, emotional, heartbreak, loss
Excited	action, thriller, adventure, adrenaline
Romantic	romance, love story, relationship
Scared	horror, suspense, supernatural
Thoughtful	sci-fi, mystery, philosophical, dystopia
Chill	slice-of-life, indie, musical

5.2 Core Function

```
mood_to_recommendations(mood: str, df: pd.DataFrame, cosine_sim: np.ndarray, top_n=10) →  
list[dict]
```

- Maps mood string → genre keywords (lookup table)
- Filters `df` for movies matching those genres
- From filtered set, picks the highest-rated movie as a seed
- Calls `get_recommendations(seed_title, df, cosine_sim, top_n)`
- Falls back to popularity-based sort if no seed found

```
MOOD_MAP = {  
    'happy': ['comedy', 'animation', 'family'],  
    'sad': ['drama'],  
    'excited': ['action', 'thriller'],  
    'romantic': ['romance'],  
    'scared': ['horror'],  
    'thoughtful': ['science fiction', 'mystery'],  
    'chill': ['music', 'indie'],  
}
```

```
def mood_to_recommendations(mood, df, cosine_sim, top_n=10):
    genres = MOOD_MAP.get(mood.lower(), [])
    mask = df['genres'].str.lower().apply(
        lambda g: any(tag in g for tag in genres)
    )
    filtered = df[mask].sort_values('vote_average', ascending=False)
    if filtered.empty:
        return df.sort_values('popularity', ascending=False).head(top_n).to_dict('records')
    seed = filtered.iloc[0]['title']
    return get_recommendations(seed, df, cosine_sim, top_n)
```

6. Low-Level Design: preprocess.py

Steps

1. **Load raw CSV** — read `movies_raw.csv` using pandas
2. **Drop duplicates** — deduplicate on `title` + `release_year`
3. **Handle nulls** — fill NaN in text columns with empty string; drop rows missing `title`
4. **Parse JSON columns** — TMDb-style datasets store `genres`, `cast`, `keywords` as JSON strings; parse and extract names
5. **Normalize text** — lowercase, remove special characters
6. **Feature engineering** — call `build_feature_soup()` and attach result as `soup` column
7. **Export** — write `movies_clean.csv`

7. EDA Notebook — Key Analyses (eda.ipynb)

Analysis	Purpose
Genre distribution bar chart	Identify dominant genres in dataset

Analysis	Purpose
Rating histogram	Check score distribution, skewness
Popularity vs vote_average scatter	Spot outliers & correlation
Missing value heatmap	Guide imputation strategy
TF-IDF top terms per genre	Validate soup quality
Similarity score distribution	Ensure matrix isn't degenerate
Mood coverage check	Verify each mood maps to $\geq N$ movies

8. API Contract (consumed by backend)

Endpoint: **GET /recommend/content**

Param	Type	Description
<code>title</code>	<code>string</code>	Movie title to base recommendations on
<code>mood</code>	<code>string</code>	Mood keyword (alternative to title)
<code>top_n</code>	<code>int</code>	Number of results (default: 10)

Response:

```
{
  "source": "content",
  "seed": "Inception",
  "recommendations": [
    {"title": "Interstellar", "score": 0.87, "genres": "sci-fi"},
    {"title": "The Matrix", "score": 0.81, "genres": "sci-fi, action"}
  ]
}
```

9. Error Handling

Scenario	Handling
Movie title not in dataset	Return <code>404</code> with message: <code>"Title not found"</code>
Mood not in MOOD_MAP	Fall back to popularity-ranked top-N
Model artifacts missing	Raise startup error, log and exit
Empty filtered results	Fallback to global popularity sort

10. Performance Considerations

- **Precompute at startup:** Load `.pkl` artifacts once; avoid recomputing cosine matrix per request
- `linear_kernel` over `cosine_similarity`: ~2× faster for sparse TF-IDF matrices
- **Top-N slicing:** Only retrieve and sort top-N scores, not all N^2 pairs
- **Dataset size target:** Tested on TMDB 5000 (~4,800 movies); scales to ~50K before memory becomes a concern

11. Dependencies

```
pandas>=1.5
scikit-learn>=1.2
numpy>=1.23
pickle (stdlib)
fastapi or flask    # consumed by backend member
```

12. Integration Points with Other Members

Member	Interface
Backend (Member 2/3)	Calls <code>/recommend/content</code> REST endpoint; passes <code>title</code> or <code>mood</code>
Collaborative ML (Member 5)	Results can be blended/re-ranked with CF scores at the API layer

Member	Interface
Frontend (Member 1)	Mood selector UI sends selected mood tag to backend → forwarded to this engine